

PartnerAPI

UML Design Package

Software Engineering Project

October 6, 2026

Contents

1	Architecture Overview	2
2	Use Case Diagram	2
3	Class Diagram	2
4	Sequence Diagram	3
5	Activity Diagram	3
6	Booking State Diagram	3
7	Cohesion	3
8	Coupling	3
9	Design Rationale	3
10	Conclusion	4

1 Architecture Overview

PartnerAPI separates the public partner interface from the internal booking platform. The architecture follows the principle:

“Public API clients should depend on stable public contracts rather than internal booking-system implementation details.”

This separation reduces coupling between external partner integrations and internal booking-system implementation.

2 Use Case Diagram

The use case model identifies the primary interactions between partner agencies, administrators, and the PartnerAPI platform.

Primary use cases:

- Authenticate
- Search Hotels
- View Availability
- Create Booking
- Retrieve Booking
- Cancel Booking
- Access Sandbox
- Manage API Keys
- View Usage Reports

3 Class Diagram

The class diagram represents the primary domain objects.

Key classes include:

- Partner
- APIKey
- Hotel
- Room
- Booking
- RateLimitPolicy
- UsageRecord

4 Sequence Diagram

The sequence diagram describes the “Search and Book a Hotel Through the API” scenario.

5 Activity Diagram

The activity diagram details the partner booking workflow.

6 Booking State Diagram

A booking progresses through controlled states during its lifecycle.

7 Cohesion

The system uses high-cohesion modules.

- Authentication module handles authentication.
- Booking module handles booking lifecycle logic.
- Rate-limit module handles request limits.
- Reporting module handles usage data.
- Partner module handles partner configuration.

Each module has a focused responsibility, which ensures high cohesion.

8 Coupling

The public API is separated from the internal booking implementation.

The API communicates through stable service interfaces rather than directly depending on internal database structures. This allows the internal booking platform to evolve without forcing partners to change their integrations unnecessarily.

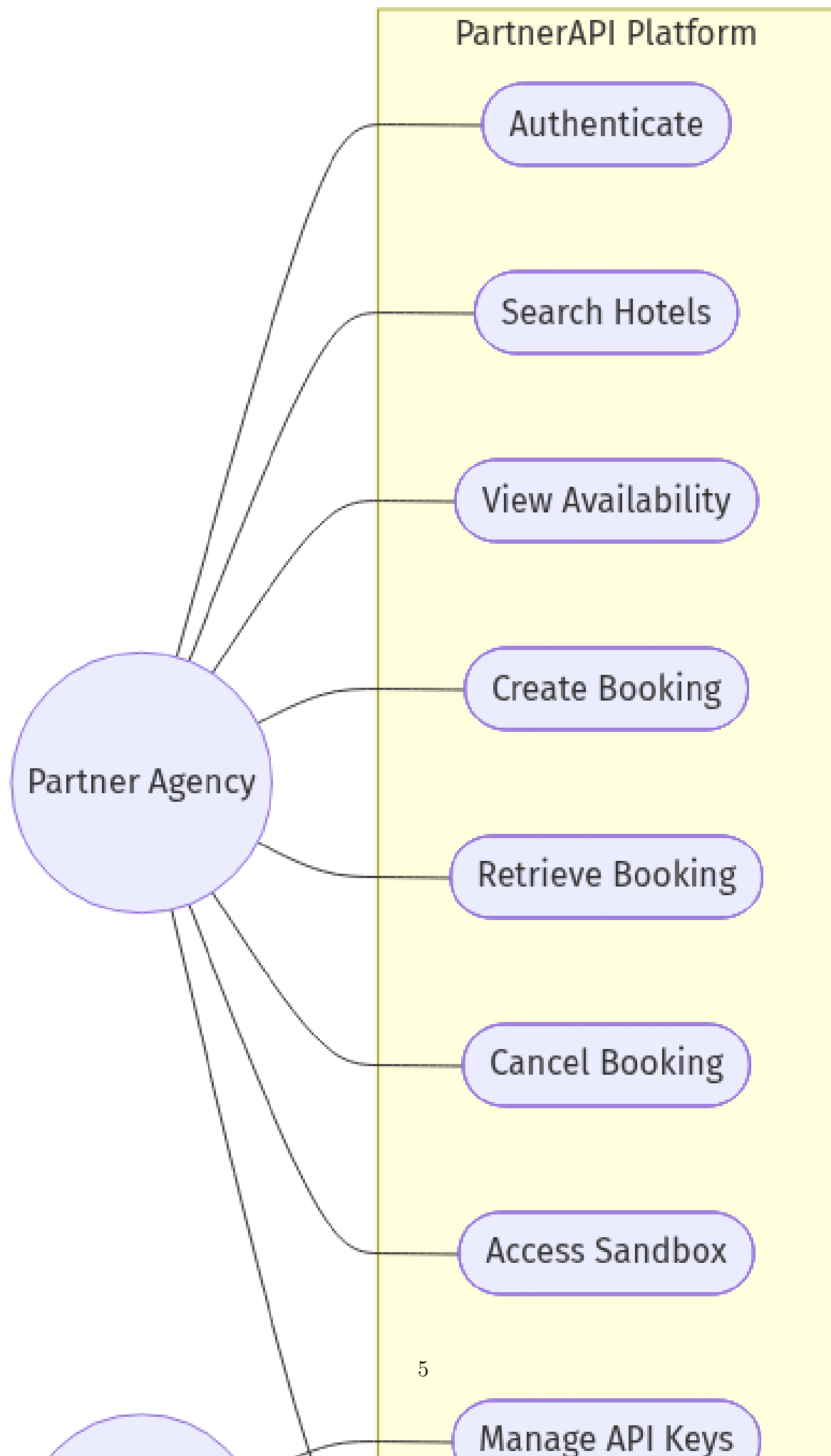
9 Design Rationale

The design aims for:

- High cohesion
- Low coupling
- Stable API contracts
- Clear responsibility boundaries
- Independent testing
- Controlled change

10 Conclusion

The UML design package defines the architecture and behavior of PartnerAPI. This separation of concerns guarantees stability, performance, and clear boundaries between the internal booking system and external partner integrations.



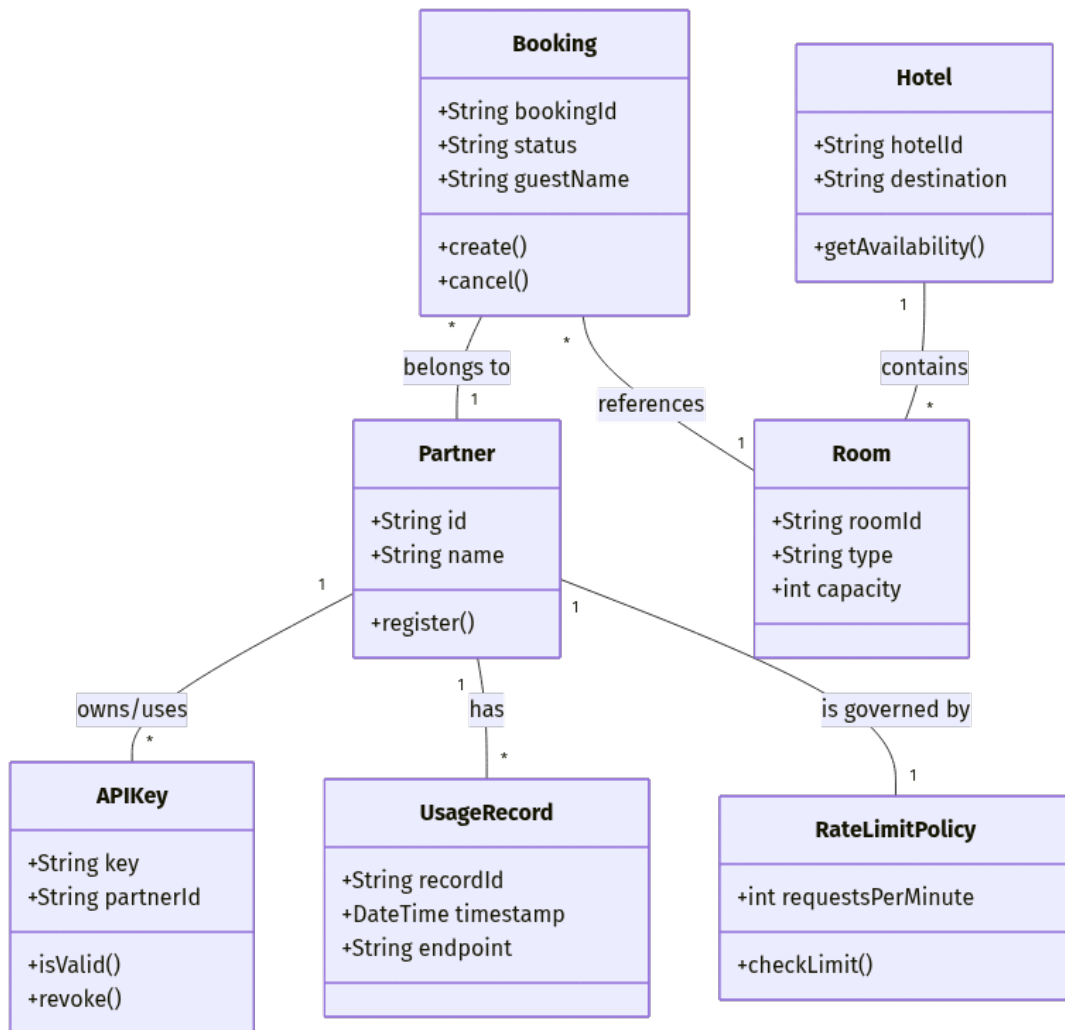


Figure 2: PartnerAPI Class Diagram

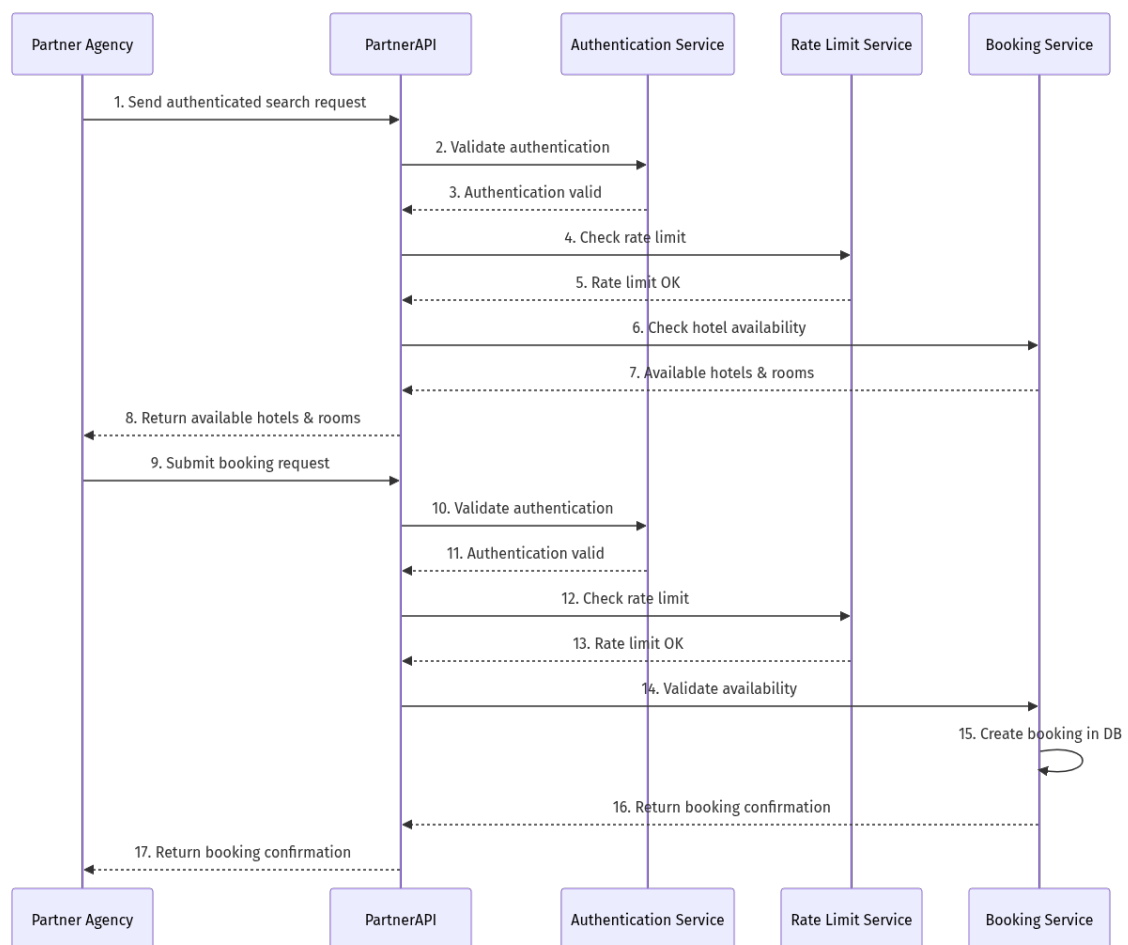


Figure 3: Search and Book Hotel Sequence

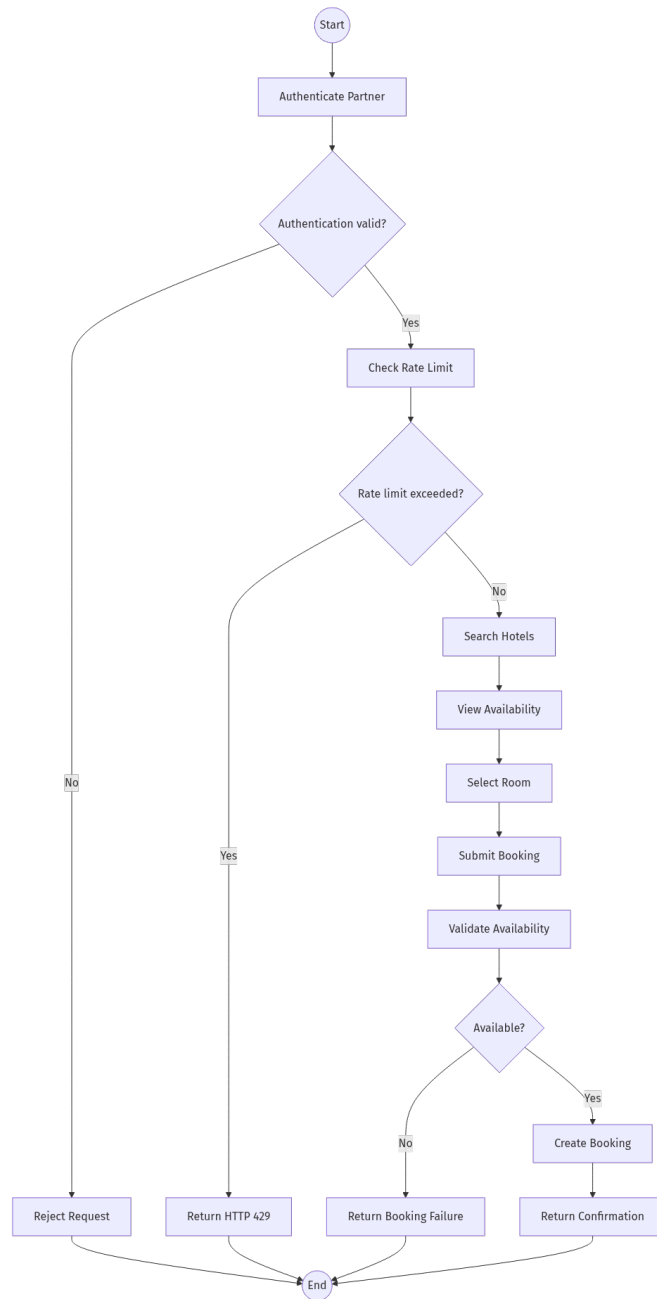


Figure 4: Partner Booking Activity

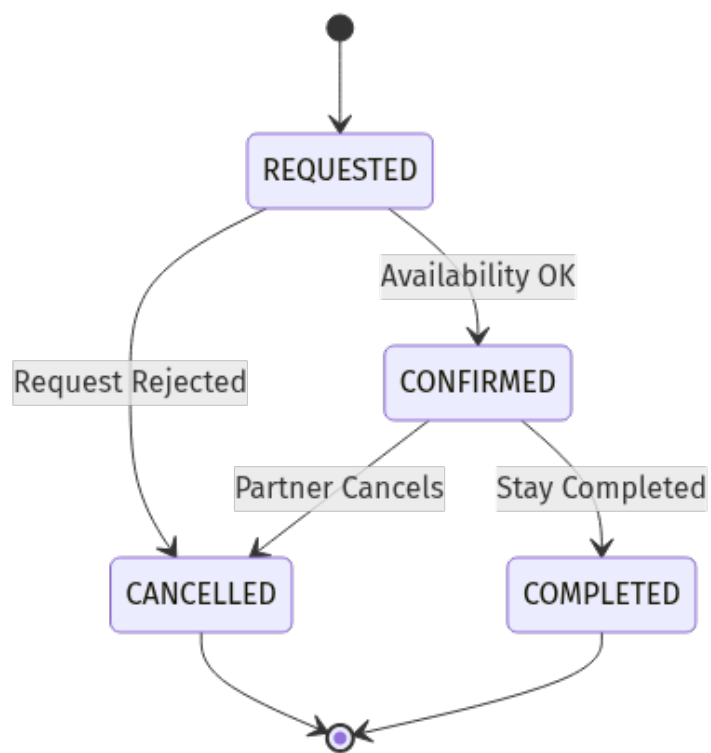


Figure 5: Booking State Diagram